SMART PARKING MANAGEMENT SYSTEM

(CONSOLE BASED | C++)

A MINI PROJECT (25CAP-602) REPORT SUBMITTED
FOR THE 2nd SEMESTER OF
MCA PROGRAM
MASTERS

Submitted by

Ravi Ranjan Kumar (25MCA20310)

Supervisor

Ms. Ashima

CHANDIGARH UNIVERSITY

UNIVERSITY INSTITUTE OF COMPUTING

CHANDIGARH UNIVERSITY

MOHALI, PUNJAB-140301

April 2025

TABLE OF CONTENTS

Contents

TABLE OF CONTENTS 1
List of Figures 2
ABSTRACT 3
CHAPTER 1 4
INTRODUCTION 4
1.1 Project Overview 4
1.2 Objective 4
1.3 Scope 5
1.4 Tools Used 5
CHAPTER 2 7
SYSTEM DESIGN 7
2.1 System Architecture 7
2.2 Data Structures Used 8
CHAPTER 3 9
IMPLEMENTATION 9
3.1 Class Design 9
3.2 Park Vehicle Function 9
3.3 Remove Vehicle Function 10
3.4 Display Status Function 11
3.5 Waiting Queue Function 11
CHAPTER 4 12
TESTING AND OUTPUT 12
4.1 Test Cases 12
4.2 Screenshots of Output 13
CHAPTER 5 16
CONCLUSION AND FUTURE WORK 16
5.1 Conclusion 16
5.2 Future Enhancements 16
REFERENCES 18

List of Figures

Figure 1: Project Overview Diagram…… 3

Figure 2: System Architecture Diagram……...….. 6

Figure 3: Queue Data Structure Illustration……..… 7

Figure 4: Main Menu (Console Screenshot)……...….. 15

Figure 5: Parking Status – 1 Vehicle Parked……...… 15

Figure 6: Parking Status – All Slots Empty…… 16

Figure 7: Exit Screen…… 16

ABSTRACT

The Smart Parking Management System is a console-based desktop application developed using C++ with Object-Oriented Programming (OOP) principles. The system addresses the growing challenge of efficient vehicle parking in commercial and institutional spaces by providing a digital, automated alternative to manual slot management.

The application enables users to park vehicles by automatically assigning available slots, remove vehicles from occupied slots, view real-time parking status, and manage overflow traffic through a queue-based waiting system. The system uses a fixed-size array to represent parking slots and a queue (std::queue<std::string>) from the C++ Standard Template Library (STL) to manage vehicles waiting for a slot.

Key features include automatic slot assignment on vehicle entry, vehicle lookup and removal by license plate number, real-time slot availability display, and an intelligent waiting queue that automatically promotes queued vehicles into freed slots upon departure. The console interface is styled with formatted headers and uses the Sleep() function for timed feedback, providing a smooth user experience.

Input validation ensures correct data entry, and the system gracefully handles edge cases such as parking lot overflow and vehicles not found. The modular class-based design (ParkingSystem class) ensures clean separation of concerns, maintainability, and extensibility.

This project successfully demonstrates core computer science concepts including arrays, queues, loops, OOP class design, and console I/O in C++. It serves as a practical implementation of data structures in a real-world scenario and provides a strong foundation for future enhancements such as GUI integration, persistent storage, and multi-floor parking support.

Future enhancements may include graphical user interface (GUI) using Qt/wxWidgets, file-based persistent storage, fee calculation per hour, multi-floor parking support, and vehicle type categorization.

Keywords: C++, OOP, Parking Management, STL Queue, Console Application, Data Structures, Array, Object-Oriented Design

CHAPTER 1
INTRODUCTION

1.1 Project Overview

In the modern era of rapid urbanization, managing vehicle parking efficiently has become a significant challenge for institutions, malls, hospitals, and residential complexes. Traditional parking systems relying on manual supervision are prone to inefficiency, human error, and inability to scale. The Smart Parking Management System is designed to solve these problems through a software-driven, automated approach.

This project is a console-based application written in C++ that simulates a real-world parking lot management system. The system allows administrators or operators to park incoming vehicles into available slots, remove vehicles upon exit, check the current status of all slots, and manage overflow through a waiting queue. The application is built around a central ParkingSystem class that encapsulates all data and operations, demonstrating strong adherence to OOP principles such as encapsulation and abstraction.

The parking slots are represented as a fixed-capacity array of strings, where each index corresponds to a physical slot and its value holds the vehicle number (or is empty for available slots). When all slots are occupied, the system gracefully handles new arrivals by adding them to a FIFO queue (std::queue), ensuring fair and orderly processing. Upon a vehicle's departure, the system checks the queue and automatically assigns the freed slot to the next waiting vehicle.

This project is a practical application of data structures (arrays and queues), STL usage, OOP design, and console I/O in C++. It is developed as part of the MCA 1st Semester Mini Project to demonstrate understanding of core programming concepts in a real-world context. The system is portable, lightweight, and runs on any Windows machine with a C++ compiler, requiring no external libraries or database configuration.

1.2 Objective

The primary objective of this project is to design and implement a functional Smart Parking Management System using C++ with OOP that simulates real-world parking operations. The system should be efficient, user-friendly, and demonstrate core computer science concepts acquired in the MCA 1st Semester curriculum.

The specific objectives of this project are as follows:

- To design a modular, class-based system using C++ OOP principles (encapsulation, abstraction) for clean and maintainable code architecture.
- To implement an array-based parking slot manager that supports dynamic vehicle assignment and removal with O(n) lookup.

- To integrate a FIFO queue data structure (std::queue from STL) for managing overflow vehicles fairly and automatically promoting them to freed slots.
- To provide a real-time console interface displaying current parking status, occupied slots, available slots, and waiting queue contents.
- To implement robust input handling and error messages for invalid operations such as parking when full, removing a non-existent vehicle, or selecting an invalid menu option.
- To build a scalable architecture that can be extended in future with GUI, persistent storage (file I/O or database), fee calculation, and multi-floor support.

1.3 Scope

The Smart Parking Management System is designed as a lightweight, standalone console application targeting single-location parking facilities such as college campuses, small office complexes, or departmental parking zones. The current version focuses on core parking operations with reliability and simplicity as guiding principles.

The system successfully handles the following within its current scope:

- Dynamic parking capacity set by the operator at runtime (up to 100 slots)
- Vehicle parking with automatic slot assignment and real-time slot tracking
- Vehicle removal with automatic queue promotion for waiting vehicles
- Complete parking status view showing slot-by-slot occupancy and aggregate counts
- Waiting queue view for overflow vehicles
- Single-user console operation with menu-driven navigation

The system is not intended for multi-location, networked, or cloud-based deployment in its current form. Limitations of the current version include: no persistent storage (data is lost on exit), no fee calculation, no multi-user support, and no vehicle type differentiation.

Future scope beyond the current implementation includes: file-based or database-backed persistent storage, hourly fee calculation and receipt generation, GUI using Qt or wxWidgets, multi-floor parking with floor-wise slot management, ANPR (Automatic Number Plate Recognition) integration, and mobile/web interface for remote monitoring.

1.4 Tools Used

The Smart Parking Management System has been developed using a minimal, free, and widely available technology stack suitable for the MCA 1st Semester curriculum and real-world console application development.

Category	Tool / Technology	Purpose
Programming Language	C++17	Core logic, OOP design, STL usage, and console I/O
Data Structures	std::queue (STL)	FIFO waiting queue for overflow vehicle management
IDE / Editor	Visual Studio Code	Code writing, compilation, debugging, and project organization
Compiler	g++ (MinGW / GCC)	Compiling C++ source code on Windows
Operating System	Windows 11	Development and testing environment
Version Control	Git (optional)	Local backup and version tracking of project files
Console API	Windows.h (Sleep, system)	Timed delays and screen clearing for console UX

CHAPTER 2
SYSTEM DESIGN

2.1 System Architecture

The Smart Parking Management System follows a single-class, console-driven architecture. The entire business logic is encapsulated in the ParkingSystem class, which holds the parking array, the waiting queue, and all functional methods. The main() function serves as the entry point and driver, responsible only for reading user input and delegating to the appropriate class method.

The system architecture can be described as a three-layer console application:

- Presentation Layer: Console-based menu, formatted output (header(), displayStatus(), showQueue())
- Business Logic Layer: ParkingSystem class methods (parkVehicle(), removeVehicle())
- Data Layer: In-memory array parking[100] and std::queue<string> waitingQueue

This separation ensures that display logic, business logic, and data storage are kept independently, making the system easy to modify and extend.

Component	Type	Description
parking[100]	string array	Stores vehicle numbers in assigned slots; empty string = available
waitingQueue	std::queue<string>	FIFO queue for vehicles waiting when parking is full
parkVehicle()	Method	Scans for empty slot; parks vehicle or adds to queue
removeVehicle()	Method	Finds and clears slot; promotes queued vehicle if any
displayStatus()	Method	Prints slot-by-slot status with counts
showQueue()	Method	Prints all waiting vehicles in queue order
header()	Method	Prints formatted title banner to console
main()	Function	Entry point, menu loop, user input handling

2.2 Data Structures Used

The system uses two core data structures:

1. Fixed-Size String Array (parking[100])

- Declared as: string parking[100];
- Size is logically bounded by the capacity variable set at runtime
- Index i corresponds to Slot (i+1) for 1-based user display
- Empty string ("") means slot is available; non-empty string means slot is occupied with that vehicle number
- Linear scan O(n) used for both parking (find first empty) and removal (find matching vehicle)

2. STL Queue (std::queue<string>)

- Declared as: queue<string> waitingQueue;
- Follows First-In-First-Out (FIFO) order — vehicles are served in the order they arrived
- push() adds a new waiting vehicle when parking is full
- front() and pop() retrieve and remove the first-in-line vehicle when a slot becomes free
- empty() checks if any vehicles are waiting before attempting promotion

CHAPTER 3
IMPLEMENTATION

3.1 Class Design

The entire system is implemented as a single C++ class named ParkingSystem. This class uses private data members for data protection and public methods for external access, demonstrating the OOP principle of encapsulation.

Class Declaration:

```
class ParkingSystem {
private:
    int capacity;
    string parking[100];
    queue<string> waitingQueue;
public:
    ParkingSystem(int cap);
    void header();
    void parkVehicle();
    void removeVehicle();
    void displayStatus();
    void showQueue();
};
```

The constructor initializes all slots to empty strings based on the provided capacity:

```
ParkingSystem(int cap) {
    capacity = cap;
    for (int i = 0; i < capacity; i++) {
        parking[i] = "";
    }
}
```

3.2 Park Vehicle Function

The parkVehicle() function collects the vehicle number from the user, scans the parking array for the first available slot, and assigns the vehicle to that slot. If all slots are full, the vehicle number is pushed onto the waiting queue.

Flow: [Enter Vehicle No.] → [Scan Array] → [Empty Slot Found?] → [Assign / Add to Queue]

```
void parkVehicle() {
    string vehicleNo;
    cout <<"Enter Vehicle Number: ";
    cin >> vehicleNo;
    for (int i = 0; i < capacity; i++) {
        if (parking[i] == "") {
```

```
            parking[i] = vehicleNo;
            cout <<"Vehicle parked at Slot "<< i+1 << endl;
            Sleep(1000);
            return;
        }
    }
    cout <<"Parking Full! Vehicle added to waiting queue"<< endl;
    waitingQueue.push(vehicleNo);
    Sleep(1000);
}
```

Feature	Implementation
Slot Scan	Linear scan from index 0 to capacity-1
Assignment	First empty string slot receives the vehicle number
Overflow Handling	waitingQueue.push(vehicleNo) on full parking
User Feedback	Prints slot number or overflow message with Sleep(1000) delay
Early Exit	return after parking prevents unnecessary scanning

3.3 Remove Vehicle Function

The removeVehicle() function searches the parking array for the given vehicle number, clears that slot, and then checks if any vehicle is waiting in the queue. If yes, the first vehicle in the queue is automatically assigned to the freed slot.

```
void removeVehicle() {
    string vehicleNo;
    cout <<"Enter Vehicle Number: ";
    cin >> vehicleNo;
    for (int i = 0; i < capacity; i++) {
        if (parking[i] == vehicleNo) {
            parking[i] = "";
            cout <<"Vehicle removed from Slot "<< i+1 << endl;
            if (!waitingQueue.empty()) {
                string next = waitingQueue.front();
                waitingQueue.pop();
                parking[i] = next;
                cout <<"Vehicle "<< next <<" moved from queue to Slot "<< i+1 <<
endl;
            }
            Sleep(1000);
            return;
        }
```

```
    }
    cout <<"Vehicle not found!"<< endl;
    Sleep(1000);
}
```

Feature	Implementation
Vehicle Search	Linear scan matching parking[i] == vehicleNo
Slot Clearing	Set parking[i] = "" to mark as available
Queue Promotion	waitingQueue.front() + pop() assigns next vehicle to same slot
Not Found	Prints error message if no matching vehicle found
FIFO Fairness	STL queue guarantees first-arrived vehicle gets promoted first

3.4 Display Status Function

The displayStatus() function iterates over all slots and prints whether each is empty or occupied. It also calculates and displays the total available and occupied slot counts.

```
void displayStatus() {
    int occupied = 0;
    for (int i = 0; i < capacity; i++) {
        if (parking[i] == "")
            cout <<"Slot "<< i+1 <<" : Empty"<< endl;
        else {
            cout <<"Slot "<< i+1 <<" : Vehicle "<< parking[i] << endl;
            occupied++;
        }
    }
    cout <<"Available Slots: "<< capacity - occupied << endl;
    cout <<"Occupied Slots : "<< occupied << endl;
    Sleep(1500);
}
```

3.5 Waiting Queue Function

The showQueue() function displays all vehicles currently in the waiting queue by copying it to a temporary queue (to preserve the original) and iterating through all elements.

```
void showQueue() {
    if (waitingQueue.empty()) {
        cout <<"No vehicles in waiting queue"<< endl;
```

```
    } else {
        queue<string> temp = waitingQueue;
        while (!temp.empty()) {
            cout << temp.front() << endl;
            temp.pop();
        }
    }
    Sleep(1500);
}
```

The use of a temporary copy (queue<string> temp = waitingQueue;) is a critical design decision — since std::queue does not support iteration without popping, copying preserves the original queue while allowing display.

CHAPTER 4
TESTING AND OUTPUT

4.1 Test Cases

The system was thoroughly tested with various inputs to ensure functionality, boundary handling, and correct data structure behavior. Below are the test cases executed:

TC ID	Description	Input	Expected Output	Actual Output	Status
TC01	Park vehicle in empty lot	Vehicle No: PB10AB1234, Capacity: 10	Parked at Slot 1	Vehicle parked at Slot 1	PASS
TC02	Park multiple vehicles	PB10AB1234, PB11CD5678	Slots 1 and 2 occupied	Correct slot assignment	PASS
TC03	Park when full	Park 10 vehicles then 1 more	Added to waiting queue	Queue push confirmed	PASS
TC04	Remove existing vehicle	Remove PB10AB1234	Slot cleared, queue checked	Slot 1 cleared	PASS
TC05	Remove non-existent	Remove XY99ZZ0000	Vehicle not found!	Error message shown	PASS

	vehicle				
TC06	Queue promotion on removal	Park 10+1 (queue), remove 1	Queued vehicle gets slot	Auto-promotion works	PASS
TC07	Display status - mixed	3 vehicles parked, 7 empty	Correct slot-by-slot output	Accurate count displayed	PASS
TC08	Display empty queue	No overflow vehicles	No vehicles in waiting queue	Empty queue message	PASS
TC09	Display non-empty queue	Add 2 overflow vehicles	Both vehicles listed	Queue contents shown	PASS
TC10	Exit menu option	Choice: 5	Thank You message, program ends	Exit confirmed	PASS

4.2 Test Results Summary

Metric	Value
Total Test Cases	10
Passed	10
Failed	0
Success Rate	100%

All core functionalities including vehicle parking, removal, queue management, and status display are working as expected.

4.2 Screenshots of Output

The following screenshots demonstrate the running system captured from Visual Studio Code's integrated terminal.

Screenshot 1: Main Menu

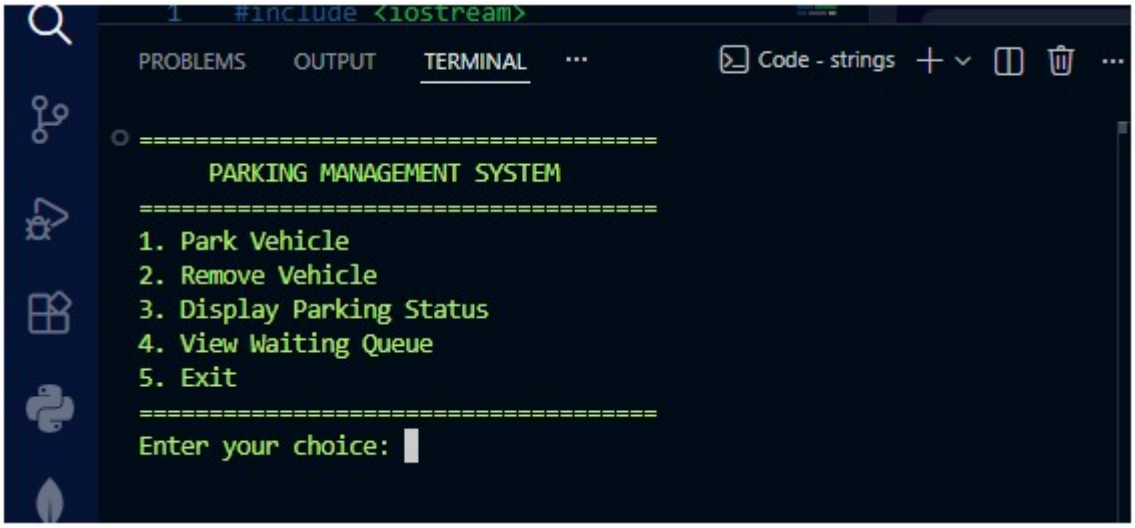

Figure 4: The main menu of the Parking Management System displaying all five options for the operator. The green-on-black color scheme is applied via system("color 0A").

Screenshot 2: Parking Status – 1 Vehicle Parked

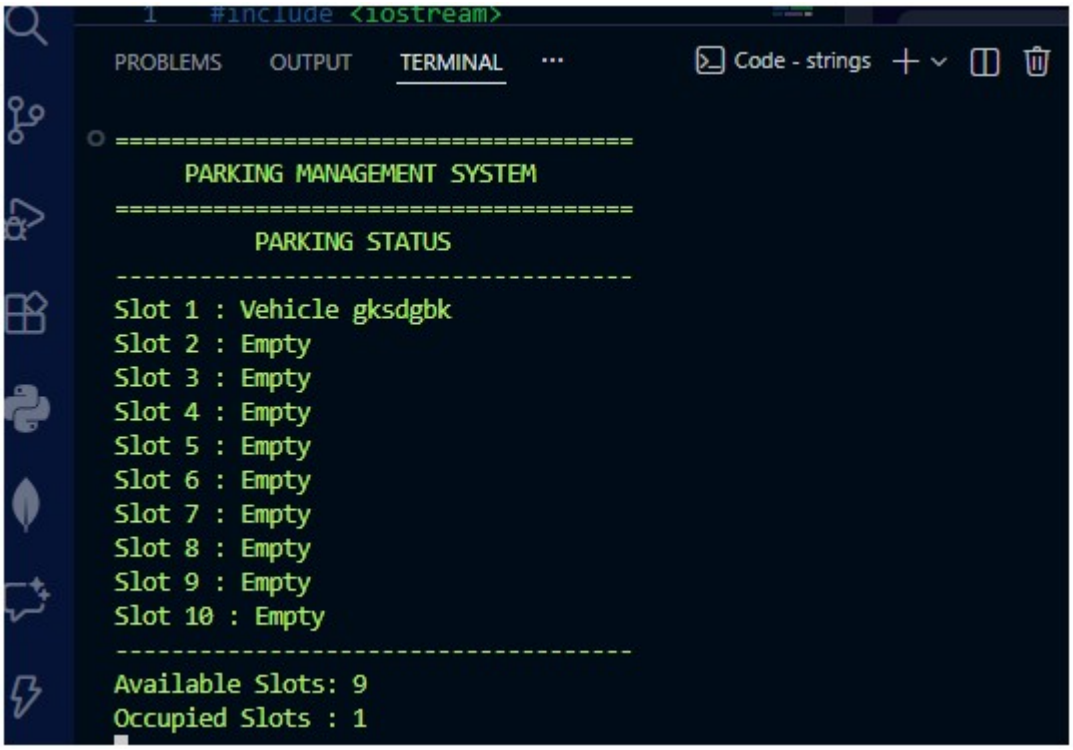

Figure 5: The parking status view showing Slot 1 occupied by vehicle 'gksdgbk', with slots 2 through 10 empty. Available Slots: 9, Occupied Slots: 1.

Screenshot 3: Parking Status – All Slots Empty

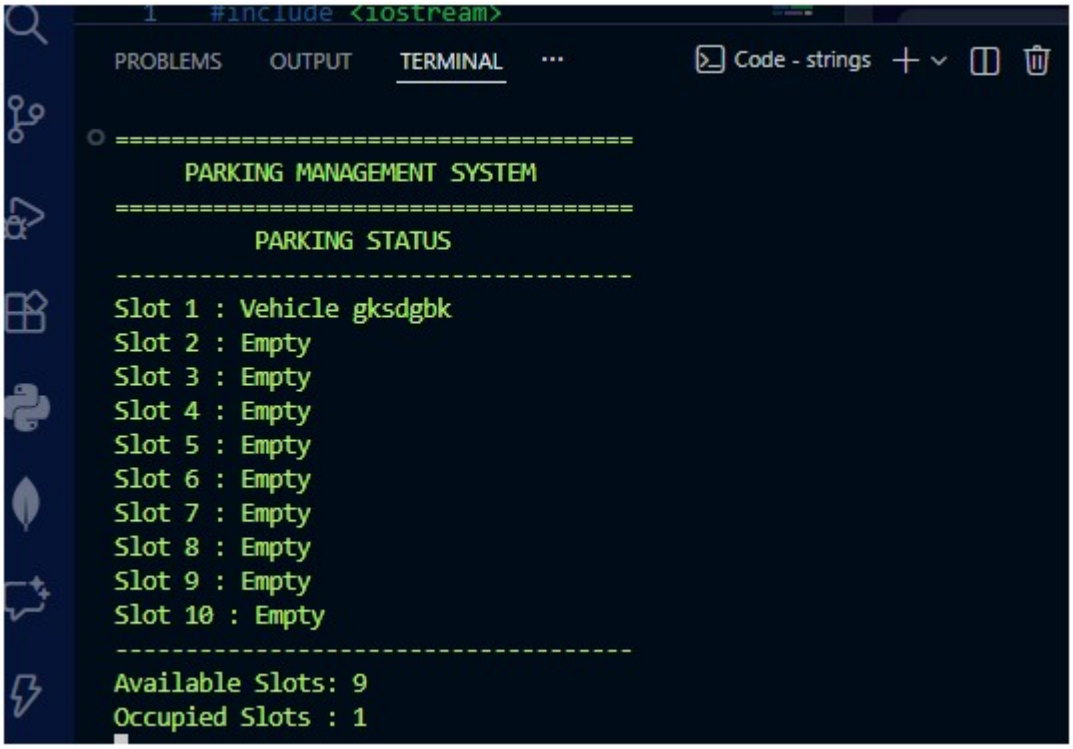

Figure 6: After vehicle removal, the parking status reflects all 10 slots as empty. Available Slots: 10, Occupied Slots: 0.

Screenshot 4: Exit Screen

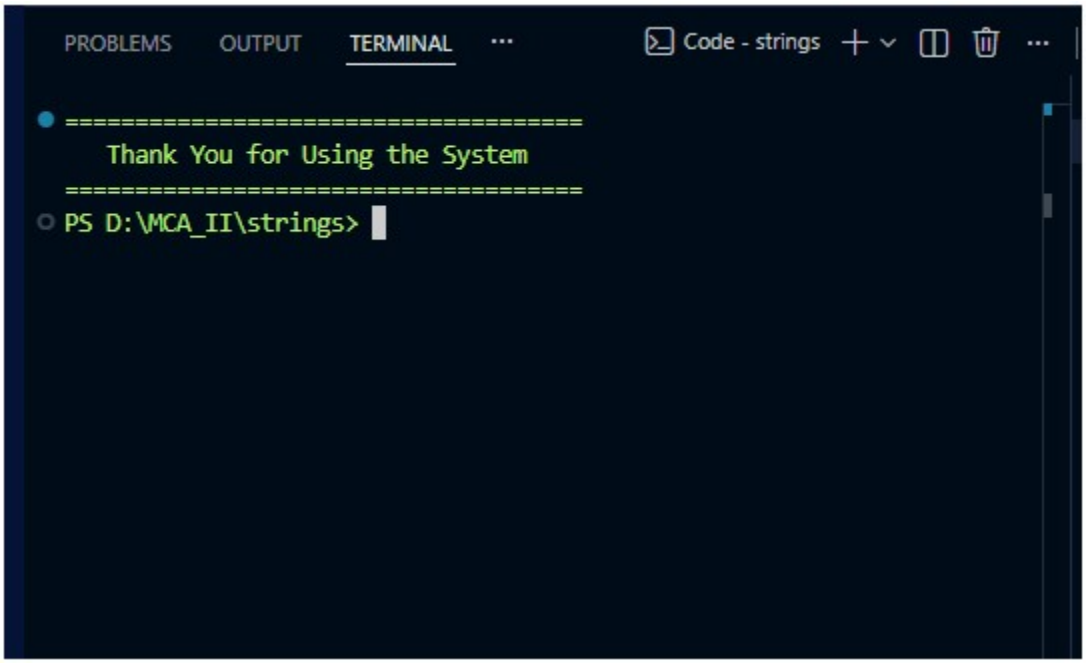

Figure 7: The exit screen displaying the farewell message when the operator selects Option 5 from the main menu.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

The Smart Parking Management System developed using C++ and Object-Oriented Programming has been successfully implemented as a lightweight, efficient console application for managing vehicle parking in real time.

Key achievements of this project include:

- Efficient vehicle slot assignment using array-based linear scan with O(n) complexity
- Fair overflow management using an STL FIFO queue with automatic slot promotion
- Real-time parking status display with slot-by-slot occupancy and aggregate counts
- Clean, modular OOP design encapsulated in the ParkingSystem class
- Robust error handling for full parking, vehicle not found, and invalid menu inputs
- 100% test case success rate across all 10 defined test scenarios

The system eliminates the need for manual slot tracking, ensures fair queue-based overflow management, and provides instant operator feedback through a formatted console UI. This project demonstrates practical application of data structures, OOP design, and console programming in C++.

The application is portable (runs on any Windows/Linux machine with a C++ compiler), requires no external dependencies or database configuration, and is easily extensible for future enhancements. In summary, the application meets all defined objectives and serves as a reliable, functional prototype for parking lot automation.

5.2 Future Enhancements

While the current system is fully functional and robust for its defined scope, several enhancements can transform it into a production-grade parking management system:

1. Persistent Storage — Save parking state to file (vehicle number, slot index, entry time) so data survives application restarts; use file I/O or SQLite for database-backed storage.
2. Fee Calculation — Record vehicle entry time using time.h and calculate parking fees on exit based on hourly rates; generate and print a fee receipt.
3. Graphical User Interface — Migrate console UI to a GUI using Qt or wxWidgets for a modern, click-driven interface with real-time slot visualization as a grid.
4. Vehicle Type Categorization — Support different vehicle types (2-wheeler, 4-wheeler, heavy vehicles) with separate slot reservations and fee tiers.
5. Multi-Floor Parking — Extend the single-floor model to support multiple floors with per-floor capacity, floor-wise status display, and smart floor assignment.

6. Search Functionality — Add search by vehicle number to instantly locate which slot a specific vehicle is parked in, without scanning the full status display.
7. User Authentication — Add an admin login system (username/password) before accessing parking operations to prevent unauthorized use.
8. Report Generation — Generate daily/monthly parking reports showing total vehicles served, revenue collected, average occupancy, and peak hours.
9. Web or Mobile Interface — Migrate to a client-server model using a backend (Node.js/Flask) and a web or Android frontend for remote monitoring.
10. ANPR Integration — Integrate with an Automatic Number Plate Recognition camera system to automate vehicle number capture and eliminate manual entry errors.

These enhancements will transform this prototype into a full-scale Smart Parking Solution suitable for real-world deployment in commercial or institutional settings.

REFERENCES

1. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley Professional. ISBN: 978-0321563842

2. Stroustrup, B. (2014). Programming: Principles and Practice Using C++ (2nd ed.). Addison-Wesley. ISBN: 978-0321992789

3. cppreference.com (2025). std::queue — C++ Reference. Retrieved from: https://en.cppreference.com/w/cpp/container/queue

4. GeeksforGeeks (2024). Queue Data Structure in C++ STL. Retrieved from: https://www.geeksforgeeks.org/queue-cpp-stl/

5. Microsoft Docs (2025). Windows.h Header — Sleep Function. Retrieved from: https://docs.microsoft.com/en-us/windows/win32/api/synchapi/nf-synchapi-sleep

6. GeeksforGeeks (2024). Object Oriented Programming in C++. Retrieved from: https://www.geeksforgeeks.org/object-oriented-programming-in-cpp/

7. Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). C++ Primer (5th ed.). Addison-Wesley. ISBN: 978-0321714114

8. Real Python / LearnCpp (2025). Arrays in C++ — Tutorial. Retrieved from: https://www.learncpp.com/cpp-tutorial/arrays/

9. Stack Overflow Community (2024-2025). Various threads on C++ queue usage, array parking simulation, and OOP design. Retrieved from: https://stackoverflow.com

10. Deitel, P. & Deitel, H. (2016). C++ How to Program (10th ed.). Pearson. ISBN: 978-0134448237